Manuale d'uso
Sistema Audit Gate

BYTE-SWARMY — Quality gate per progetti software

Versione 1.0

Novembre 2026

Adatto a periti informatici junior e sviluppatori

1. Introduzione

Il **Sistema Audit Gate** di BYTE-SWARMY è un quality gate automatico che analizza un progetto software alla ricerca di problemi di sicurezza, qualità del codice e configurazione, **prima che il progetto venga distribuito o eseguito**. Il sistema è progettato per essere configurabile, riusabile su qualsiasi progetto Python e integrato sia nell'interfaccia grafica sia nell'orchestratore automatico.

Il gate controlla sette categorie di problemi: variabili inutilizzate, codice morto, porte esposte su interfacce pubbliche, comandi sospetti che potrebbero creare tunnel o reverse proxy verso l'esterno, import di moduli potenzialmente pericolosi (socket, ctypes, pickle, ecc.), esistenza e integrità dei file critici del sistema, e validità del file di configurazione TOML. Ogni categoria produce un risultato PASS, FAIL o SKIP, e l'aggregato determina il gate globale.

Il sistema è rivolto a **sviluppatori e periti informatici** che vogliono una verifica automatica della qualità e sicurezza del proprio codice prima della consegna o della messa in produzione. Può essere usato sia su BYTE-SWARMY stesso (self-audit con 7 categorie) sia su qualsiasi progetto Python esterno (audit con 5 categorie attive e 2 SKIP, perché le categorie file-critici e config.toml sono specifiche di BYTE-SWARMY).

Il vantaggio principale rispetto a tool come pylint o bandit è l'integrazione nativa con l'orchestratore: in modalità strict, il gate può bloccare l'esecuzione dei task su progetti che non superano il controllo, trasformando l'audit da strumento diagnostico a vera e propria policy di sicurezza enforceable.

■■ Il sistema funziona sia su BYTE-SWARMY stesso (self-audit) sia su qualsiasi progetto Python esterno. Vedi sezione 9 per esempi pratici di entrambi gli scenari.

2. Architettura del sistema

Il sistema è composto da **quattro componenti modulari** che comunicano attraverso una API Python ben definita. Ogni componente può essere usato indipendentemente dagli altri, ma insieme formano una pipeline completa di quality assurance.

2.1 Componenti

Il primo componente è la **CLI** (`tests/test_monthly_audit.py`), che esegue le sette scansioni AST-based sul progetto target e produce un report in formato testo, JSON o HTML. Accetta i flag `--target PATH` per puntare a un progetto esterno, `--gate` per un output compatto una-riga-per-categoria, `--json` per output machine-readable e `--html PATH` per generare un report HTML leggibile dal browser.

Il secondo componente è l'**helper riusabile** (`CORE/audit_gate.py`), che incapsula la logica di esecuzione dell'audit in una funzione Python `check_audit_gate(project_path)` ritornando un oggetto **GateResult** con stato PASS/FAIL, modalità, categorie dettagliate e finding completi. Questo helper gestisce anche il salvataggio/caricamento dello stato del gate in `<project>/.audit_status.json` per permettere alla GUI di mostrare l'ultimo risultato noto senza rilanciare l'audit ogni volta.

Il terzo componente è il **pannello GUI** (`GUI/audit_project_panel.py`), accessibile dalla Plancia di BYTE-SWARMY tramite la card viola '■ Audit Progetto'. Offre un ComboBox per scegliere il progetto target, un pulsante per lanciare l'audit in background (QThread, non blocca la GUI), un badge grande PASS/FAIL e un tree espandibile con i dettagli per categoria.

Il quarto componente è il **gate bloccante nell'orchestratore** (`CORE/orchestrator.py`), che prima di eseguire ogni task tramite `worker.execute()` lancia `check_audit_gate()` sul progetto target. In modalità strict, se il gate è FAIL il task viene marcato come 'blocked:audit_gate' e non viene eseguito; in modalità advisory viene solo loggato un warning ma il task procede.

2.2 Flusso di esecuzione

Il flusso tipico è: l'utente (o l'orchestratore) invoca `check_audit_gate(project_path)`; l'helper legge la configurazione da `CONFIG/config.toml` (sezione `[audit.gates]`) per determinare la modalità (off/advisory/strict); lancia la CLI `test_monthly_audit.py --target PATH --json` come subprocess per isolamento (se la CLI crasha, non porta giù l'orchestratore); parsifica il JSON risultante; costruisce un GateResult; lo salva su disco per riuso futuro; ritorna al chiamante.

> ■■ L'helper usa subprocess per isolamento: se l'audit crasha per un bug o un file Python malformato, l'orchestratore non viene interrotto. Questo è il principio fail-open: meglio non bloccare il lavoro che bloccare tutto per un falso positivo dell'audit.

3. Le 7 categorie di audit

Ogni categoria è una scansione AST-based indipendente che cerca pattern specifici nel codice Python del progetto target. Le categorie si dividono in due gruppi: **advisory** (solo warning, non influenzano il gate) e **bloccanti** (critical, determinano il PASS/FAIL del gate).

Categoria	Tipo	Cosa rileva	Self	Esterno
■ Variabili inutilizzate	advisory	Variabili pubbliche a livello modulo mai referenziate	✓	✓
■ Codice morto	advisory	Funzioni/metodi pubblici mai chiamati nel codice	✓	✓
■■ Porte esposte	■ bloccante	bind() su 0.0.0.0, ::, HTTPServer, uvicorn su host non-loopback	✓	✓
■ Tunnel/Reverse proxy	■ bloccante	ngrok, cloudflared, ssh -R, socat, docker -p, kubectl port-forward, ecc.	✓	✓
■ Import banditi	■ bloccante	socket, ctypes, pickle, http.server fuori allowlist	✓	✓
■ File critici	■ bloccante	Esistenza e non-vuotezza di orchestratore, security, config, ecc.	✓	– SKIP
■■ config.toml valido	■ bloccante	Sintassi TOML valida del file di configurazione	✓	– SKIP

3.1 Distinzione self-audit vs esterno

Le ultime due categorie (File critici e config.toml valido) sono **specifiche di BYTE-SWARMY**: controllano l'esistenza dei file essenziali dell'orchestratore (CORE/orchestrator.py, SAFETY/security.py, GUI/control_center.py, ecc.) e la validità del config.toml di BYTE-SWARMY. Quando si audita un progetto esterno, queste due categorie vengono marcate SKIP perché non hanno senso fuori dal contesto di BYTE-SWARMY. Il gate globale tiene conto di questo: le categorie SKIP non influenzano il PASS/FAIL.

3.2 Severità: critical vs warning

Ogni finding ha una severità: **critical** per le categorie bloccanti (porte esposte, tunnel, import banditi, file mancanti), **warning** per le categorie advisory (variabili inutilizzate, codice morto) e per i casi borderline delle categorie bloccanti (es. porte su loopback ma non standard). Il gate globale è FAIL se almeno una categoria bloccante ha critical > 0; è PASS altrimenti. I warning non bloccano mai il gate ma vengono mostrati nel report per consentire pulizia del codice.

4. Modalità di funzionamento

Il gate supporta tre modalità operative, configurabili nel file `CONFIG/config.toml` nella sezione `[audit.gates]`. La scelta della modalità dipende dal livello di enforcement desiderato: si consiglia di iniziare con advisory, passare a strict solo quando il progetto è stabilmente pulito, e usare off solo per disabilitare temporaneamente il gate in caso di debug.

4.1 Modalità off (disattivata)

In modalità **off** il gate è completamente disattivato: `check_audit_gate()` ritorna sempre PASS senza eseguire alcuna scansione. Questa modalità è utile quando si vuole temporaneamente bypassare il gate (es. durante il debug di un problema urgente) senza modificare il codice dell'orchestratore. L'audit è comunque disponibile manualmente dalla CLI e dalla GUI.

CONFIG/config.toml

```
[audit.gates]
enabled = false
mode = "off"
```

4.2 Modalità advisory (raccomandata per iniziare)

In modalità **advisory** il gate viene eseguito normalmente, ma se il risultato è FAIL viene solo loggato un warning nell'orchestratore e il task procede comunque. È la modalità consigliata per iniziare perché permette di vedere quali problemi esistono nel progetto senza bloccare il lavoro. Si consiglia di tenere advisory per almeno 2-4 settimane, risolvere i critical trovati, e solo allora passare a strict.

CONFIG/config.toml

```
[audit.gates]
enabled = true
mode = "advisory"
target = "auto"
```

4.3 Modalità strict (bloccante)

In modalità **strict** il gate è bloccante: se il risultato è FAIL, il task viene marcato come `blocked:audit_gate` e non viene eseguito. L'utente deve risolvere i critical (porte esposte, tunnel, import banditi, file mancanti) prima di poter procedere. Questa modalità è indicata per progetti in produzione o quando si vuole enforceare una policy di sicurezza rigida. Il fallimento del gate viene salvato nello stato del task e visualizzato nella GUI.

CONFIG/config.toml

```
[audit.gates]
enabled = true
mode = "strict"
target = "auto"
categories = []  # vuoto = tutte le bloccanti attive
```

■■ In modalità strict, se l'audit crasha per un errore interno (es. file Python malformato che fa fallire il parser AST), l'orchestratore usa il principio fail-open: logga un warning ma procede comunque. Questo evita che un bug dell'audit blocchi tutto il lavoro.

5. Uso da riga di comando (CLI)

La CLI `tests/test_monthly_audit.py` è l'entry point principale del sistema. Può essere invocata direttamente da terminale o tramite il launcher `./byte-swarm.sh audit`. Supporta quattro modalità di output e due opzioni di targeting.

5.1 Self-audit (default)

Senza argomenti o con `--gate`, la CLI esegue l'audit su BYTE-SWARMY stesso (7 categorie attive, inclusi file critici e config.toml):

```
# Output compatto gate mode (una riga per categoria)
python tests/test_monthly_audit.py --gate

# Output JSON machine-readable
python tests/test_monthly_audit.py --json

# Genera report HTML
python tests/test_monthly_audit.py --html logs/audit/report.html --no-text

# Tramite launcher
./byte-swarm.sh audit
```

5.2 Audit di un progetto esterno

Con `--target PATH` la CLI punta a una directory di progetto esterna. Le categorie file-critici e config.toml vengono automaticamente marcate SKIP:

```
# Audit di un progetto in project/
python tests/test_monthly_audit.py --target project/mio_progetto --gate

# Audit di un progetto arbitrario sul disco
python tests/test_monthly_audit.py --target /home/user/myapp --gate

# Output JSON + HTML insieme
python tests/test_monthly_audit.py --target /path/to/proj \
    --json --html /tmp/report.html
```

5.3 Esempio di output --gate

L'output di `--gate` è compatto e leggibile, una riga per categoria con status, conteggio critical e warning. Il lucchetto ■ indica le categorie bloccanti:

```
======================================================================
  AUDIT GATE — byte-swarmy (self)
  2026-11-03T12:08:42+00:00
======================================================================

  CATEGORIA                       STATUS   CRIT  WARN
  ------------------------------ -------- ----- -----
     ■ Variabili inutilizzate     ✓ PASS   0     1
     ■ Codice morto               ✓ PASS   0     116
  ■ ■■ Porte esposte               ✓ PASS   0     1
  ■ ■ Tunnel/Reverse proxy       ✓ PASS   0     0
```

```
■ ■ Import banditi             ✓ PASS   0     0
■ ■ File critici               ✓ PASS   0     0
■ ■■ config.toml valido         ✓ PASS   0     0

GATE: ✓ PASS — nessuna categoria bloccante con critical
Totale finding: 118 (critical: 0, warning: 118)
```

5.4 Exit code

La CLI ritorna exit code 0 se il gate è PASS, 1 se è FAIL. Questo permette di usarla in script bash, CI/CD pipeline e git hooks per bloccare automaticamente commit o deploy se il gate non passa:

```
# Esempio: blocca il commit se il gate fallisce
python tests/test_monthly_audit.py --target . --gate || \
    echo "ERRORE: audit gate FALLITO, commit bloccato" && exit 1
```

6. Uso dalla GUI

Il pannello GUI 'Audit Progetto' è accessibile dalla Plancia di BYTE-SWARMY tramite la sesta card (viola, in basso a destra). È l'interfaccia più user-friendly per eseguire audit manuali su progetti specifici senza dover ricordare i comandi CLI.

6.1 Apertura del pannello

- Avvia la GUI con `./byte-swarm.sh gui`
- Dalla Plancia di controllo (schermata iniziale con 10 card), individua la card viola '■ Audit Progetto' con badge 'Gate'
- Click sulla card → si apre il pannello AuditProjectPanel

6.2 Selezione del progetto target

In alto al pannello c'è un ComboBox 'Progetto target' che elenca automaticamente: ■ **byte-swarmy (self)** per il self-audit, e una voce ■ **nome** per ogni sottocartella di `project/`. Per aggiungere un nuovo progetto al ComboBox ci sono due modi: usare il tab ■ in alto (apre un file dialog per scegliere una cartella), oppure copiare manualmente la cartella del progetto dentro `~/Desktop/byte-swarmy/project/` e cliccare il pulsante ■ (refresh) a destra del ComboBox.

6.3 Esecuzione dell'audit

- Seleziona il progetto dal ComboBox
- Click sul pulsante verde '■ Esegui audit gate'
- Il pulsante diventa '■ Audit in corso...' e si disabilita
- L'audit gira in background (QThread, ~2-3 secondi per progetti piccoli)
- La GUI resta reattiva durante l'audit
- Al completamento: badge grande PASS (verde) o FAIL (rosso) + tree categorie

6.4 Lettura dei risultati

Il **badge GATE PASS/FAIL** in alto al pannello mostra il risultato aggregato: verde con ✓ PASS se nessuna categoria bloccante ha critical, rosso con ✗ FAIL altrimenti. Sotto il badge c'è il conteggio totale (critical, warning, mode). Il **tree categorie** sotto mostra una riga per ognuna delle 7 categorie con icona, nome, status, critical count, warning count e descrizione. Le categorie bloccanti sono precedute dal lucchetto ■ e in grassetto. Le categorie FAIL vengono auto-espanso per mostrare i finding dettagliati (file:riga + messaggio).

✓ Lo stato dell'ultimo audit viene salvato in <project>/.audit_status.json. Quando riapri il pannello e selezioni un progetto già auditato, vedi immediatamente l'ultimo risultato senza dover rilanciare

l'audit. Per aggiornare, click di nuovo su 'Esegui audit gate'.

6.5 Report HTML

Il pulsante '■ Apri report HTML' apre il browser sul report HTML completo dell'ultimo audit del progetto selezionato. Il report HTML è più dettagliato del tree GUI: mostra tutti i finding (non solo i primi 30 per categoria) con tabella navigabile, badge colorati per severity, e distribuzione per tipo. I report vengono salvati in `logs/audit/<YYYY-MM-DD>_audit.html` con symlink `latest.html` al più recente.

7. Gate bloccante nell'orchestratore

L'integrazione del gate nell'orchestratore è la caratteristica più potente del sistema: trasforma l'audit da strumento diagnostico a vera policy di sicurezza enforceable. Quando il gate è attivo in modalità strict, l'orchestratore si rifiuta di eseguire task su progetti che non superano il controllo.

7.1 Quando viene eseguito il gate

Il gate viene eseguito **prima di ogni worker.execute(task)** nel ciclo principale dell'orchestratore (`CORE/orchestrator.py`, metodo `_tick()`). Più precisamente, dopo la pianificazione (Planner/GenericTaskPlanner) e la selezione del modello, ma prima di passare il task al Worker per l'esecuzione ReAct. Questo significa che il gate viene valutato per ogni task, non una sola volta all'avvio.

7.2 Determinazione del project_path

Il project_path da auditare viene determinato in questo modo: se il config dice `target = "auto"` (default), l'orchestratore usa la directory `project/` di BYTE-SWARMY come target, a meno che il task non abbia un `context.project_path` esplicito (es. task creati dal tab di progetto specifico). Se invece il config ha un path esplicito (es. `target = "/home/user/myapp"`), quello viene usato sempre, indipendentemente dal task.

7.3 Comportamento per modalità

Modalità	Gate PASS	Gate FAIL	Audit crasha
off	skip (sempre PASS)	n/a (mai eseguito)	n/a
advisory	log info, procedi	log warning, procedi	fail-open, procedi
strict	log info, procedi	BLOCK: mark failed	fail-open, procedi

7.4 Stato del task bloccato

Quando il gate blocca un task in strict mode, il task viene marcato come `failed` con reason `blocked:audit_gate (N critical findings)` dove N è il numero di critical. Il task non viene ritentato automaticamente (per evitare loop infiniti di blocchi). L'utente deve risolvere i critical (porte esposte, tunnel, import banditi, file mancanti) e poi riavviare il task manualmente dalla GUI o dalla CLI. Il log dell'orchestratore mostra il dettaglio dei critical che hanno causato il blocco.

8. Configurazione

Tutta la configurazione del gate è centralizzata nella sezione `[audit.gates]` del file `CONFIG/config.toml`. Non è necessario modificare codice Python per attivare/disattivare il gate o cambiarne la modalità: basta editare il TOML e riavviare l'orchestratore.

8.1 Sezione completa

CONFIG/config.toml

```
# ═══════════════════════════════════════════════════════════════
# AUDIT GATES — Quality gate per progetti target
# ═══════════════════════════════════════════════════════════════
[audit.gates]
enabled = false              # true|false — attiva/disattiva il gate
mode = "advisory"            # off|advisory|strict
target = "auto"              # auto = progetto del tab attivo, oppure path
categories = []              # vuoto = tutte le bloccanti attive
```

8.2 Parametri dettagliati

Parametro	Tipo	Default	Descrizione
enabled	bool	false	Attiva (true) o disattiva (false) il gate. Se false, mode viene ignorato.
mode	string	"advisory"	off = skip, advisory = log warning ma procedi, strict = blocca il task
target	string	"auto"	auto = usa il progetto del tab attivo, oppure path esplicito (es. /home/user/app)
categories	array	[]	Lista opzionale di categorie bloccanti da considerare. Vuoto = tutte.

8.3 Esempio: configurazione strict completa

```
[audit.gates]
enabled = true
mode = "strict"
target = "auto"
# categories vuoto = tutte le bloccanti attive:
#   exposed_ports, potential_tunnel, banned_import,
#   critical_files, config_validity
categories = []
```

8.4 Verifica della configurazione

Per verificare che la configurazione sia valida TOML e che il gate sia correttamente attivo, usare:

```
# Verifica sintassi TOML
python -c "import tomllib; print(tomllib.load(open('CONFIG/config.toml','rb')))"

# Verifica configurazione gate
python -c "from CORE.audit_gate import load_gate_config; print(load_gate_config())"
```

```
# Output atteso:
# {'enabled': True, 'mode': 'strict', 'target': 'auto', 'categories': []}
```

9. Esempi pratici

9.1 Scenario A — Audit di byte-swarmy su se stesso

Questo è lo scenario più comune: verificare che BYTE-SWARMY stesso sia sano. Si esegue periodicamente (es. mensilmente via scheduler) per cogliere regressioni. Il self-audit attiva tutte e 7 le categorie, incluse file-critici e config.toml che sono specifici di BYTE-SWARMY.

```
# Dalla CLI
python tests/test_monthly_audit.py --gate

# Dalla GUI: Plancia → ■ Audit Progetto →
#   ComboBox: "■ byte-swarmy (self)" →
#   Click "■ Esegui audit gate"
```

Output tipico (progetto sano):

```
  CATEGORIA                        STATUS   CRIT  WARN
  -------------------------------- -------- ----- -----
     ■ Variabili inutilizzate     ✓ PASS   0     1
     ■ Codice morto               ✓ PASS   0     116
  ■ ■■ Porte esposte               ✓ PASS   0     1
  ■ ■ Tunnel/Reverse proxy        ✓ PASS   0     0
  ■ ■ Import banditi              ✓ PASS   0     0
  ■ ■ File critici                ✓ PASS   0     0
  ■ ■■ config.toml valido          ✓ PASS   0     0

  GATE: ✓ PASS — nessuna categoria bloccante con critical
```

I warning (116 codice morto + 1 variabile + 1 porta non-standard) sono accettabili: non bloccano il gate ma indicano opportunità di pulizia. Il gate è PASS perché nessuna categoria bloccante ha critical.

9.2 Scenario B — Audit di un progetto con problemi

Mettiamo di star scrivendo un nuovo programma `mio_app.py` con questo codice insicuro:

mio_app.py

```
import socket
import pickle  # ■ pericoloso: deserializzazione non sicura
import os

def main():
    s = socket.socket()
    s.bind(("0.0.0.0", 8080))  # ■ espone su tutte le interfacce
    s.listen(5)

def funzione_mai_usata():
    pass  # ■ dead code

if __name__ == "__main__":
    main()
```

Mettiamo il file in `project/mio_progetto/` e lanciamo l'audit:

```
python tests/test_monthly_audit.py --target project/mio_progetto --gate
```

Output atteso (gate FAIL):

```
  CATEGORIA                        STATUS   CRIT  WARN
  -------------------------------- -------- ----- -----
     ■ Variabili inutilizzate     ✓ PASS   0     0
     ■ Codice morto               ✓ PASS   0     1
  ■ ■■ Porte esposte               ✗ FAIL   1     1
  ■ ■ Tunnel/Reverse proxy       ✓ PASS   0     0
  ■ ■ Import banditi             ✓ PASS   0     2
  ■ ■ File critici               – SKIP   0     0
  ■ ■■ config.toml valido         – SKIP   0     0

  GATE: ✗ FAIL — almeno una categoria bloccante con critical

  ■■■ CRITICAL DA RISOLVERE (1) ■■■■■■■■■■■■■■■■■■■■■■■■■
  [exposed_port] mio_app.py:7
    bind su host non-loopback '0.0.0.0':8080 — servizio esposto
```

Il gate è FAIL perché la categoria 'Porte esposte' (bloccante) ha 1 critical: il bind su 0.0.0.0 espone il servizio su tutte le interfacce di rete, non solo loopback. In modalità strict, l'orchestratore bloccherebbe qualsiasi task su questo progetto. Per risolvere: cambiare `("0.0.0.0", 8080)` in `("127.0.0.1", 8080)` per ascoltare solo su loopback. I warning (1 dead code, 2 import banditi socket+pickle) non bloccano il gate ma indicano problemi di qualità/sicurezza da risolvere in seguito.

10. Risoluzione problemi (FAQ)

Q1: Il gate è troppo lento. Come ottimizzo?

L'audit AST-based è veloce (~2-3 secondi per progetti piccoli, ~10-20 secondi per progetti con centinaia di file). Se è troppo lento, probabilmente il progetto target ha una directory `.venv` o `node_modules` che viene scansionata inutilmente. Aggiungi la directory alle esclusioni in `_ENV_DIRS` in `tests/test_monthly_audit.py`. In alternativa, usa la modalità advisory invece di strict: il gate viene eseguito ma non blocca, quindi la latenza è tollerabile.

Q2: Come aggiungo una categoria custom?

Aggiungi una nuova funzione `_scan_tua_categoria(root)` in `tests/test_monthly_audit.py` che ritorna una lista di finding dict. Poi registra la categoria in `AUDIT_CATEGORIES` e la mappa `_FINDING_TYPE_TO_CATEGORY`. La nuova categoria apparirà automaticamente nella CLI, nella GUI e nel gate dell'orchestratore senza modifiche ulteriori.

Q3: Il gate blocca tutto in strict mode. Come bypasso?

Per bypassare temporaneamente il gate senza modificare config, commenta la sezione `[audit.gates]` in `CONFIG/config.toml` oppure imposta `enabled = false`, poi riavvia l'orchestratore. Per un bypass permanente, usa la modalità advisory che logga warning ma non blocca. Per risolvere il problema alla radice, identifica i critical dalla GUI (pannello Audit Progetto) e risolvili nel codice.

Q4: Dove trovo i report HTML?

I report HTML vengono salvati in `logs/audit/<YYYY-MM-DD>_audit.html`. Esiste anche un symlink `logs/audit/latest.html` che punta al report più recente. Dalla GUI, il pulsante '■ Apri report HTML' apre automaticamente il browser sul report del progetto corrente. Da CLI, usa `--html PATH` per specificare il percorso di output.

Q5: Il gate dice FAIL ma non vedo critical. Perché?

Probabilmente stai guardando l'output testuale che mostra solo i primi 20 finding. Apri il report HTML completo (pulsante '■ Apri report HTML' nella GUI) per vedere tutti i finding. In alternativa, usa `--json` e filtra con `jq '.findings[] | select(.severity == "critical")'`. Il gate è FAIL solo se almeno una categoria **bloccante** ha critical > 0: controlla le righe con ■ nel gate output.

Q6: Come disattivo il gate senza modificare config?

Non è possibile disattivare il gate senza toccare config, ma puoi impostare `mode = "off"` che ha lo stesso effetto. In alternativa, lancia l'orchestratore con una variabile d'ambiente che sovrascrive la modalità (questo richiede una piccola modifica al codice di `load_gate_config()` in `CORE/audit_gate.py` per leggere `AUDIT_GATE_MODE` env var).

11. Riferimenti

11.1 File del sistema

File	Ruolo	Righe
tests/test_monthly_audit.py	CLI + 7 scansioni AST + output --gate/--json/--html	~1060
CORE/audit_gate.py	Helper riusabile: check_audit_gate(), GateResult, save/load status	~280
GUI/audit_project_panel.py	Pannello GUI: ComboBox progetti, tree categorie, QThread worker	~560
GUI/control_center.py	Card "■ Audit Progetto" nella Plancia + registrazione detail_stack	—
CORE/orchestrator.py	Gate bloccante pre-Worker.execute (strict=block, advisory=warn)	—
CONFIG/config.toml	Sezione [audit.gates]: enabled, mode, target, categories	—
scripts/audit_mensile.sh	Bash wrapper per audit mensile: genera HTML+JSON in logs/audit/	—

11.2 Comandi utili

```
# Self-audit gate mode
python tests/test_monthly_audit.py --gate

# Audit progetto esterno
python tests/test_monthly_audit.py --target project/mio_progetto --gate

# Audit tramite launcher
./byte-swarm.sh audit

# Audit mensile automatico (genera HTML+JSON in logs/audit/)
./scripts/audit_mensile.sh

# Verifica configurazione gate
python -c "from CORE.audit_gate import load_gate_config; print(load_gate_config())"

# Test rapido helper
python -m CORE.audit_gate /path/to/project --mode strict
```

11.3 Risorse esterne

- Repository BYTE-SWARMY: `https://github.com/wildlux/byte-swarmy`
- Documentazione ReportLab (per customizzare il report HTML): `https://docs.reportlab.com`
- Specifiche MCP (per integrazione futura con server MCP esterni): `https://modelcontextprotocol.io`
- Python AST module (per capire le scansioni): `https://docs.python.org/3/library/ast.html`

Manuale generato per BYTE-SWARMY — Sistema Audit Gate v1.0 — Novembre 2026. Per aggiornamenti o domande, consulta il repository GitHub o contatta il maintainer.